ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 26

Servicios del framework

- El contenedor de dependencias
- Middleware: las capas de la cebolla
- Middleware global y grupos de rutas
- Mensajes flash de verdad
- Validación al estilo Laravel
- Autenticación
- Funciones de atajo para las vistas
- Nuestro MVC frente a Laravel



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Nuestro MVC ya tiene esqueleto: front controller, router, controladores, vistas y modelos. Hoy le ponemos los **órganos**: las piezas que hacen que un *framework* sea cómodo de usar y que en Laravel parecen magia.

- Un **contenedor de dependencias** que fabrica los objetos y les da lo que necesitan.
- **Middleware**: capas que revisan cada petición antes de que llegue al controlador (CSRF, login...).
- Un **validador** con reglas como `obligatorio|min:3`, que vuelve atrás solo con los errores.
- Mensajes ***flash*** de verdad, que duran exactamente una petición.
- **Autenticación**: login con la base de datos y rutas protegidas.

Y al final, una tabla que os enseña qué pieza de nuestro MVC corresponde a cada pieza de Laravel. Os aviso: después de esta clase, Laravel os va a parecer muy familiar. 😉

•
👌 Cómo vamos a trabajar



1
📘 El contenedor de dependencias



2
🤔 Middleware: las capas de la cebolla



3
👌 Middleware global y grupos de rutas



4
🔍 Mensajes flash de verdad



5
🚀 Validación al estilo Laravel



6
⚠️ Autenticación



7
👍 Funciones de atajo para las vistas



8
😮 Nuestro MVC frente a Laravel



9
🚀 Ejercicios



10
👋 Lo que tenéis que llevaros de hoy





CLASE 26

👌 Cómo vamos a trabajar

Sustituid el contenido de `academia-mvc` por el de `proyecto/` (conservad vuestro `.env`) y ejecutad `composer dump-autoload`. Usuario de prueba: `admin@academia.test` / `secreto123`.

Esquema

```
config/servicios.php          ← NUEVO: las «recetas» del contenedor
src/Nucleo/
├── Contenedor.php            ← NUEVO
├── Aplicacion.php            ← NUEVO: guarda el contenedor
├── Middleware.php            ← NUEVO: la interfaz de los middleware
├── Validador.php             ← NUEVO
├── ValidacionException.php   ← NUEVO
├── Auth.php                  ← NUEVO
└── Router, Peticion, Sesion y ManejadorErrores, ampliados
src/Middleware/
├── VerificarCsrf.php         ← NUEVO
└── Autenticado.php           ← NUEVO
src/Modelos/Usuario.php       ← NUEVO
src/Controladores/AuthControlador.php   ← NUEVO
vistas/auth/login.php         ← NUEVO
```



CLASE 26 · PUNTO 1

📘 El contenedor de dependencias

En la clase 22 visteis la **inyección de dependencias**: las clases reciben lo que necesitan por el constructor en lugar de fabricarlo con `new`. Bien, pero entonces… ¿quién hace los `new`? Alguien tiene que saber que para crear un `AuthControlador` primero hay que crear un `Auth`.

Ese alguien es el **contenedor de dependencias**: un almacén que sabe fabricar cada objeto [1]. Pensad en el encargado de un taller de coches: el mecánico (el controlador) le pide «una llave del 12» y el encargado se la da. El mecánico no tiene que saber de dónde sale.

Contenedor.php — php-paso-a-paso/clase26 — Visual Studio Code

Contenedor.php

```php
<?php
$contenedor = new Contenedor();

// Recetas: «cuando alguien pida un PDO, dale la conexión de BaseDatos»
$contenedor->compartido(PDO::class, fn() => BaseDatos::conexion());
$contenedor->compartido(Auth::class, fn() => new Auth());

$auth = $contenedor->obtener(Auth::class);
```

Método	Qué hace
`registrar($id, $fabrica)`	Cada vez que se pida, fabrica un objeto **nuevo**
`compartido($id, $fabrica)`	Lo fabrica la primera vez y después devuelve **siempre el mismo**
`obtener($id)`	Devuelve el objeto, siguiendo su receta o fabricándolo solo

Autowiring: el contenedor se las arregla solo

Lo mejor viene cuando **no hay receta**. El contenedor usa `Reflection`, una herramienta de PHP para que un programa examine su propio código, y **mira qué pide el constructor** [2]:

Contenedor.php — php-paso-a-paso/clase26 — Visual Studio Code

Contenedor.php

```php
<?php
class AuthControlador extends Controlador
{
    public function __construct(private Auth $auth)    // «necesito un Auth»
    {
    }
}

$controlador = $contenedor->obtener(AuthControlador::class);
// 1. ¿Hay receta para AuthControlador? No.
// 2. Miro su constructor: pide un Auth.
// 3. obtener(Auth::class) → tengo receta → se lo paso.
```

Esto se llama ***autowiring*** (autoconexión). Y es recursivo: si `Auth` pidiera a su vez otras cosas, el contenedor las fabricaría también.

El router ya no hace `new $clase()`: le pide los controladores y los middleware al contenedor. Por eso basta con escribir el constructor de `AuthControlador` para que le llegue su `Auth`.

📄 Archivos: `src/Nucleo/Contenedor.php` y `config/servicios.php`



CLASE 26 · PUNTO 2

🤔 Middleware: las capas de la cebolla

En la clase 21, cada página privada empezaba con `Sesion::requerirLogin()` y cada formulario con `Sesion::comprobarCsrf(...)`. Si se os olvidaba en una página, esa página quedaba desprotegida.

Un **middleware** es una capa que se coloca **entre la petición y el controlador**. Puede dejar pasar la petición o cortarla y responder él mismo [3]:

Esquema

```
Petición ──▶ [ VerificarCsrf ] ──▶ [ Autenticado ] ──▶ CursosControlador::guardar()
                    │                      │                        │
                ¿token válido?         ¿sesión iniciada?            ▼
                si no → 403            si no → /login           Respuesta
Respuesta ◀──────────────────────────────────────────────────────────┘
```

Todos cumplen la misma interfaz:

Middleware.php — php-paso-a-paso/clase26 — Visual Studio Code

Middleware.php

```php
<?php
interface Middleware
{
    public function procesar(Peticion $peticion, callable $siguiente): Respuesta;
}
```

Y este es el guardián de las páginas privadas, convertido en middleware:

Middleware.php — php-paso-a-paso/clase26 — Visual Studio Code

Middleware.php

```php
<?php
class Autenticado implements Middleware
{
    public function __construct(private Auth $auth)
    {
    }

    public function procesar(Peticion $peticion, callable $siguiente): Respuesta
    {
        if (!$this->auth->comprobar()) {
            Sesion::guardarMensaje("Inicia sesión para gestionar los cursos.", "error");
            return Respuesta::redirigir("/login");    // corta aquí: el controlador ni se
entera
        }
        return $siguiente($peticion);                 // deja pasar a la siguiente capa
    }
}
```

`$siguiente` es «todo lo que viene detrás»: el siguiente middleware o, al final, el controlador. El router los encadena como las capas de una cebolla:

Middleware.php — php-paso-a-paso/clase26 — Visual Studio Code

Middleware.php

```php
<?php
private function tuberia(array $middleware, callable $controlador): callable
{
    $siguiente = $controlador;
    foreach (array_reverse($middleware) as $clase) {
        $capa = $this->contenedor->obtener($clase);
        $siguiente = fn(Peticion $p) => $capa->procesar($p, $siguiente);
    }
    return $siguiente;
}
```

📄 Archivos: `src/Nucleo/Middleware.php`, `src/Middleware/Autenticado.php`, `src/Middleware/VerificarCsrf.php` y `src/Nucleo/Router.php`



💡 Un middleware también puede hacer cosas **después** del controlador, porque recibe la respuesta de `$siguiente()`. El ejercicio 01 lo aprovecha para medir cuánto tarda cada petición.



CLASE 26 · PUNTO 3

👌 Middleware global y grupos de rutas

¿Cómo se dice qué middleware se aplica a cada ruta? De dos formas:

rutas.php — php-paso-a-paso/clase26 — Visual Studio Code

rutas.php

```php
<?php
// Para TODAS las rutas: el CSRF (solo actúa en las POST)
$router->usar(VerificarCsrf::class);

// Rutas públicas
$router->get("/cursos", [CursosControlador::class, "index"]);
$router->get("/cursos/{id:\d+}", [CursosControlador::class, "ver"]);
$router->get("/login", [AuthControlador::class, "formulario"]);
$router->post("/login", [AuthControlador::class, "entrar"]);

// Un GRUPO: todas las rutas de dentro exigen haber iniciado sesión
$router->grupo([Autenticado::class], function (Router $router): void {
    $router->get("/cursos/crear", [CursosControlador::class, "crear"]);
    $router->post("/cursos", [CursosControlador::class, "guardar"]);
    // ...editar, actualizar y borrar
});
```

Leyendo `config/rutas.php` sabéis exactamente qué está protegido. Y si mañana añadís una ruta privada, basta con ponerla dentro del grupo.

📄 Archivo: `config/rutas.php`



CLASE 26 · PUNTO 4

🔍 Mensajes flash de verdad

Hasta ahora, el mensaje *flash* se borraba **al leerlo**. Si la página lo leía dos veces, la segunda ya no estaba; si no lo leía, se quedaba para la siguiente. La forma correcta es que dure **exactamente una petición**, se lea o no.

La sesión tiene ahora dos cajones:

Sesion.php — php-paso-a-paso/clase26 — Visual Studio Code

Sesion.php

```php
<?php
public static function iniciar(): void
{
    session_start();
    $_SESSION["_flash"] = $_SESSION["_flash_siguiente"] ?? [];   // lo de la petición
anterior, disponible ahora
    $_SESSION["_flash_siguiente"] = [];                          // cajón vacío para la
siguiente
}

public static function flash(string $clave, mixed $valor): void
{
    $_SESSION["_flash_siguiente"][$clave] = $valor;              // se guarda para la
PRÓXIMA petición
}
```

Así funcionan los mensajes, los errores de validación y los datos que el usuario había escrito (punto 5).

📄 Archivo: `src/Nucleo/Sesion.php`



CLASE 26 · PUNTO 5

🚀 Validación al estilo Laravel

¿Cuántos `if` de validación habéis escrito desde la clase 12? Hoy los sustituimos por **reglas**, como en Laravel [5]:

Validador.php — php-paso-a-paso/clase26 — Visual Studio Code

Validador.php

```php
<?php
public static function reglas(): array
{
    return [
        "titulo" => "obligatorio|min:3|max:120",
        "horas" => "obligatorio|entero|min:1|max:1000",
        "precio" => "obligatorio|numero|min:0|max:99999",
        "plazas" => "obligatorio|entero|min:0|max:500",
    ];
}
```

Regla	Significa
`obligatorio`	No puede estar vacío
`entero` / `numero`	Debe ser un entero / un número (admite coma decimal)
`email`	Debe ser un email válido
`min:3` / `max:120`	En textos, longitud; en números, valor

Y en el controlador, una sola línea:

Validador.php — php-paso-a-paso/clase26 — Visual Studio Code

Validador.php

```php
<?php
public function guardar(Peticion $peticion): Respuesta
{
    $datos = $peticion->validar(Curso::reglas());   // si falla, no se pasa de aquí

    $curso = (new Curso())->rellenar($datos);
    $curso->guardar();

    return $this->redirigir("/cursos/{$curso->id}", "Curso «{$curso->titulo}» creado.");
}
```

¿Qué pasa si algo no es válido? `validar()` lanza una `ValidacionException`, el manejador de errores la recoge y hace lo mismo que Laravel: **vuelve a la página anterior** con los errores y lo que el usuario había escrito, guardados como *flash*. El controlador no tiene ni un `if`.

Validador.php — php-paso-a-paso/clase26 — Visual Studio Code

Validador.php

```php
<?php
if ($e instanceof ValidacionException) {
    Sesion::flash("errores", $e->errores);
    Sesion::flash("viejo", $e->datos);
    Respuesta::redirigir($e->volverA)->enviar();
    return;
}
```

En el formulario, dos funciones de atajo muestran el error de cada campo y recuperan lo escrito:

Validador.php — php-paso-a-paso/clase26 — Visual Studio Code

Validador.php

```php
<input name="titulo" value="<?= e(viejo("titulo", $curso->titulo)) ?>">
<small class="error"><?= e(error("titulo")) ?></small>
```

`validar()` devuelve **solo** los campos que tienen reglas. Si alguien añade un campo a mano en el formulario con las herramientas del navegador, se ignora.

📄 Archivos: `src/Nucleo/Validador.php`, `src/Nucleo/Peticion.php`, `src/Controladores/CursosControlador.php` y `vistas/cursos/_formulario.php`



💡 «Volver a la página anterior» funciona gracias a la cabecera `Referer`, que el navegador envía con cada petición indicando de qué página venía. La `Peticion` la guarda en `$peticion->anterior`.



CLASE 26 · PUNTO 6

⚠️ Autenticación

El login de las clases 13, 15 y 20, convertido en un **servicio** que se puede pedir al contenedor:

Auth.php — php-paso-a-paso/clase26 — Visual Studio Code

Auth.php

```php
<?php
class Auth
{
    public function intentar(string $email, string $clave): bool { /* busca el Usuario y
comprueba la clave */ }
    public function usuario(): ?Usuario { /* el usuario conectado, o null */ }
    public function comprobar(): bool { return $this->usuario() !== null; }
    public function salir(): void { /* olvida el usuario y regenera la sesión */ }
}
```

El modelo `Usuario` hereda del modelo base, y el modelo base tiene ahora un método nuevo, `primeroDonde()`, para buscar por email: `Usuario::primeroDonde("email", $email)`.

Las rutas de login son públicas, las de gestión van en el grupo `Autenticado`, y el `AuthControlador` recibe el `Auth` por inyección:

Auth.php — php-paso-a-paso/clase26 — Visual Studio Code

Auth.php

```php
<?php
public function entrar(Peticion $peticion): Respuesta
{
    $datos = $peticion->validar(["email" => "obligatorio|email", "clave" => "obligatorio"]);

    if (!$this->auth->intentar($datos["email"], $datos["clave"])) {
        throw new ValidacionException(["email" => "Email o contraseña incorrectos."],
["email" => $datos["email"]], "/login");
    }

    return $this->redirigir("/cursos", "¡Hola, {$this->auth->usuario()?->nombre}!");
}
```

Fijaos en el truco del error de login: lanzamos nosotros una `ValidacionException`, así que vuelve al formulario por el mismo camino que cualquier otro error.

📄 Archivos: `src/Nucleo/Auth.php`, `src/Modelos/Usuario.php` y `src/Controladores/AuthControlador.php`



CLASE 26 · PUNTO 7

👍 Funciones de atajo para las vistas

Las vistas no tienen acceso al contenedor, así que lo guardamos en `Aplicacion` y añadimos unas funciones globales, igual que hace Laravel:

Función	Qué hace	En Laravel
`app(Auth::class)`	Pide un servicio al contenedor	`app()`
`auth()->comprobar()`	¿Hay alguien conectado?	`auth()->check()`
`auth()->usuario()`	El usuario conectado	`auth()->user()`
`error("titulo")`	El error de validación de un campo	`@error('titulo')`
`viejo("titulo")`	Lo que el usuario había escrito	`old('titulo')`
`campoCsrf()`	El campo oculto con el token	`@csrf`

funciones.php — php-paso-a-paso/clase26 — Visual Studio Code

funciones.php

```php
<?php if (auth()->comprobar()): ?>
    <a href="/cursos/crear">➕ Nuevo curso</a>
    <span class="usuario">👤 <?= e(auth()->usuario()->nombre) ?></span>
<?php else: ?>
    <a href="/login">Iniciar sesión</a>
<?php endif; ?>
```

📄 Archivos: `src/funciones.php` y `vistas/layout.php`

✅ Pruebas para comprobarlo

Haced esto	Debéis obtener
Ver el catálogo sin sesión	Sin botones de editar ni «Nuevo curso»
Abrir `/cursos/crear` sin sesión	Redirige a `/login` con el aviso
Entrar con un email mal escrito	Vuelve al login: «Debe ser un email válido»
Entrar con la clave incorrecta	Vuelve al login con el aviso y el email ya escrito
Entrar bien	Catálogo con «👤 Administración»
Crear un curso con título «AB», 0 horas y precio «abc»	Vuelve al formulario con un error bajo cada campo y lo escrito
Recargar ese formulario	Los errores desaparecen: el *flash* dura una petición
Crear, editar y borrar un curso	Igual que en la clase 25
Enviar un formulario con el token cambiado (F12)	403
Salir	Vuelve al catálogo sin sesión



Esta tabla es el resumen de las clases 23 a 26. Cuando abráis un proyecto de Laravel, buscad cada pieza: estará ahí, con otro nombre y muchas más opciones, pero con la misma idea [4].

Nuestro MVC	Laravel	Lo construimos en…
`public/index.php`	`public/index.php`	Clase 23
`config/rutas.php`	`routes/web.php`	Clase 24
`Router`	`Route`	Clase 24
`Peticion` / `Respuesta`	`Request` / `Response`	Clase 24
`ManejadorErrores` + `DEPURACION`	`Handler` de excepciones + `APP_DEBUG`	Clase 24
`.env` + `Entorno`	`.env` + `env()`	Clase 25
`Modelo` (Active Record)	Eloquent (`Model`)	Clase 25
`Vista` + `parcial()`	Blade + `@include`	Clase 25
`Controlador` base	`Controller`	Clase 25
`HttpException` / `noEncontrado()`	`abort(404)`	Clase 25
`Contenedor`	*Service container*	Clase 26
`config/servicios.php`	*Service providers*	Clase 26
`Middleware` + `grupo()`	*Middleware* + `Route::middleware()`	Clase 26
`$peticion->validar([...])`	`$request->validate([...])`	Clase 26
`Auth` + `auth()`	`Auth` + `auth()`	Clase 26
`viejo()` / `error()` / `campoCsrf()`	`old()` / `@error` / `@csrf`	Clase 26



Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Registro de peticiones.** Un middleware global, `RegistroPeticiones`, que apunte en `peticiones.log` cada petición: fecha, método, ruta, código de respuesta y milisegundos que ha tardado. Pista: haced vuestro trabajo **después** de llamar a `$siguiente()`. → `RegistroPeticiones.php`
- ☐ **02 · Crear cuenta.** Una página de registro con dos reglas nuevas en el validador: `igual:clave` (repetir la contraseña) y `unico:usuarios,email` (que el email no exista). Al registrarse, el usuario entra directamente. → `Validador.php`, `RegistroControlador.php` y `registro.php`
- ☐ **03 · Solo para invitados.** Un middleware `SoloInvitados`, lo contrario de `Autenticado`: si ya has iniciado sesión, `/login` y `/registro` te llevan al catálogo. Organizad las rutas en dos grupos. → `SoloInvitados.php` y `config/rutas.php`



🏆 **Reto:** las peticiones que fallan con una excepción (un 404 o una validación) no llegan a apuntarse en `peticiones.log`, porque la excepción «sale volando» antes de volver al middleware. Arreglad `RegistroPeticiones` con un `try … finally` para que se apunten todas.



CLASE 26 · PUNTO 10

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Contenedor	Sabe fabricar los objetos y les pasa sus dependencias
Autowiring	Sin receta, el contenedor mira el constructor con `Reflection` y se las apaña
Middleware	Capas entre la petición y el controlador: dejan pasar o cortan
`usar()` / `grupo()`	Middleware para todas las rutas / para un grupo de rutas
Flash	Datos que duran exactamente una petición
`validar()`	Reglas en lugar de `if`; si falla, vuelve atrás con errores y datos
`Auth`	El login como servicio: `intentar()`, `usuario()`, `comprobar()`, `salir()`
Laravel	Las mismas piezas, con más opciones: ya sabéis cómo funcionan por dentro

📚 Fuentes y documentación oficial

1. Laravel: Service Container
2. Manual de PHP: Reflection
3. Laravel: Middleware
4. Laravel: documentación oficial
5. Laravel: Validation